SQL ASSIGNMENT – ADVANCE - LMS

```
use RetailDB_3;

-- 1. Customers
CREATE TABLE Customers (
 customer_id INT AUTO_INCREMENT PRIMARY KEY,
 name VARCHAR(100),
 email VARCHAR(100),
 city VARCHAR(50),
 signup_date DATE
);

-- 2. Suppliers
CREATE TABLE Suppliers (
 supplier_id INT AUTO_INCREMENT PRIMARY KEY,
 supplier_name VARCHAR(100),
 contact_email VARCHAR(100),
 city VARCHAR(50)
);

-- 3. Shippers
CREATE TABLE Shippers (
 shipper_id INT AUTO_INCREMENT PRIMARY KEY,
 shipper_name VARCHAR(100),
 contact VARCHAR(100)
);
```

```
-- 4. Payment Methods
CREATE TABLE Payment_Methods (
 payment_id INT AUTO_INCREMENT PRIMARY KEY,
 payment_type VARCHAR(50) UNIQUE
);

-- 5. Products
CREATE TABLE Products (
 product_id INT AUTO_INCREMENT PRIMARY KEY,
 product_name VARCHAR(100),
 category VARCHAR(50),
 price DECIMAL(10,2),
 stock_qty INT,
 supplier_id INT,
 FOREIGN KEY (supplier_id) REFERENCES Suppliers(supplier_id)
);

-- 6. Orders (normalized, no total_amount column)
CREATE TABLE Orders (
 order_id INT AUTO_INCREMENT PRIMARY KEY,
 customer_id INT,
 order_date DATE,
 payment_id INT,
 shipper_id INT,
 FOREIGN KEY (customer_id) REFERENCES Customers(customer_id),
 FOREIGN KEY (payment_id) REFERENCES Payment_Methods(payment_id),
 FOREIGN KEY (shipper_id) REFERENCES Shippers(shipper_id)
);
```

```
-- 7. Order_Items
CREATE TABLE Order_Items (
 order_item_id INT AUTO_INCREMENT PRIMARY KEY,
 order_id INT,
 product_id INT,
 quantity INT,
 price_each DECIMAL(10,2),
 FOREIGN KEY (order_id) REFERENCES Orders(order_id),
 FOREIGN KEY (product_id) REFERENCES Products(product_id)
);



SELECT * FROM Customers;
SELECT * FROM Suppliers;
SELECT * FROM Shippers;
SELECT * FROM Payment_Methods;
SELECT * FROM Products;
SELECT * FROM Orders;
SELECT * FROM Order_Items;



-- Task: Solve all the below given questions by writing effective SQL queries.


a) Joins, Group By, Order By, Aggregations:


1. Find the total revenue collected by each shipper.


SELECT s.shipper_id, s.shipper_name, SUM(oi.quantity*oi.price_each) AS total_revenue
FROM Shippers s
```

```
JOIN  Orders o
ON s.shipper_id = o.shipper_id
JOIN Orders_items oi
ON o.order_id = oi.order_id
GROUP BY s.shipper_id, s.shipper_name
ORDER BY total_revenue DESC;
```

2. Show the top 5 highest-spending customers along with their total payments.

```
SELECT c.customer_id, c.name AS customer_name, SUM(o.total_amount) AS total_spent
FROM Customers c
JOIN Orders o
ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name
ORDER BY total_spent DESC
LIMIT 5;
```

3. Find product categories where the average selling price is greater than 8000.

```
SELECT category
FROM Products
GROUP BY category
HAVING AVG(price) > 8000;
```

4. Show the total number of orders placed per city, sorted by highest to lowest.

```
SELECT c.city, COUNT(o.order_id) AS total_no_of_orders
FROM Customers c
```

```
JOIN Orders o
ON c.customer_id = o.customer_id
GROUP BY c.city
ORDER BY total_no_of_orders DESC;
```

5. Find suppliers who supply more than 1 product category.

```
SELECT s.supplier_name , COUNT(DISTINCT p.category) AS count_category
FROM Suppliers s
JOIN Products p
ON p.supplier_id = s.supplier_id
GROUP BY s.supplier_name
HAVING count_category > 1;
```

6. Show each order along with the number of items it contains.

```
SELECT o.order_id, COUNT(oi.product_id) AS item_count
FROM Orders o
JOIN Order_items oi
ON o.order_id = oi.order_id
GROUP BY o.order_id
ORDER BY item_count DESC;
```

b) Subqueries – Nested & Correlated:

7. Find customers who have spent more than the average spending of all customers.

```
SELECT c.customer_id, c.name,
   SUM(oi.quantity * oi.price_each) AS total_spent
```

```
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id
JOIN Order_items oi ON o.order_id = oi.order_id
GROUP BY c.customer_id, c.name
HAVING total_spent > (SELECT AVG(total)
   FROM (
      SELECT customer_id,
         SUM(oi.quantity * oi.price_each) AS total
      FROM Orders o
      JOIN Order_items oi ON o.order_id = oi.order_id
      GROUP BY customer_id
   ) AS avg_spending
);
```

8. List all products whose price is higher than the average product price in their category.

```
SELECT product_id, product_name, category, price
FROM Products p
WHERE price > (
   SELECT AVG(price)
   FROM Products
   WHERE category = p.category
);
```

9. Show customers who placed at least one order with total_amount greater than 50,000.

```
SELECT DISTINCT c.customer_id, c.name AS customer_name
```

```
FROM Customers c
JOIN Orders o
ON c.customer_id = o.customer_id
WHERE EXISTS (
   SELECT 1
   FROM Order_items oi
   WHERE oi.order_id = o.order_id
   GROUP BY oi.order_id
   HAVING SUM(oi.price_each * oi.quantity) > 50000
);
```

10.Find customers who placed more orders than the average number of orders per customer.

```
SELECT customer_id, name, order_count
FROM (
   SELECT c.customer_id, c.name, COUNT(o.order_id) AS order_count
   FROM Customers c
   JOIN Orders o ON c.customer_id = o.customer_id
   GROUP BY c.customer_id, c.name
) AS customer_orders
WHERE order_count > (
   SELECT AVG(order_count)
   FROM (
      SELECT COUNT(o.order_id) AS order_count
      FROM Orders o
      GROUP BY o.customer_id
   ) AS avg_orders
);
```

11. Find the most expensive product(s) in the catalog.

```
SELECT product_id, product_name, category, price
FROM Products
WHERE price = (
   SELECT MAX(price)
   FROM Products
);
```

c) Window Functions:

12.Rank customers based on their total spending.

```
SELECT
   c.customer_id,
   c.name,
   SUM(oi.price_each * oi.quantity) AS total_spent,
   RANK() OVER (ORDER BY SUM(oi.price_each * oi.quantity) DESC) AS spending_rank
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id
JOIN Order_items oi ON o.order_id = oi.order_id
GROUP BY c.customer_id, c.name
ORDER BY total_spent DESC;
```

13.Find cumulative sales amount by order date.

```
SELECT
   o.order_date,
```

```
    SUM(oi.price_each * oi.quantity) AS daily_sales,
    SUM(SUM(oi.price_each * oi.quantity)) OVER (ORDER BY o.order_date) AS cumulative_sales
FROM Orders o
JOIN Order_items oi ON o.order_id = oi.order_id
GROUP BY o.order_date
ORDER BY o.order_date;
```

14.Get each customer's order count and show their percentage contribution.

```
SELECT
    c.customer_id,
    c.name,
    COUNT(o.order_id) AS order_count,
    ROUND(
        COUNT(o.order_id) * 100.0 /
        (SELECT COUNT(*) FROM Orders),
        2
    ) AS percentage_contribution
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name
ORDER BY percentage_contribution DESC;
```

15.Show the most recent order per customer.

```
SELECT o.customer_id, c.name, o.order_id, o.order_date
FROM Orders o
JOIN Customers c ON o.customer_id = c.customer_id
WHERE o.order_date = (
```

```
    SELECT MAX(o2.order_date)
    FROM Orders o2
    WHERE o2.customer_id = o.customer_id
);
```

16.List each product with its sales quantity and rank products within each category by total sales.

```
SELECT
    p.product_id,
    p.product_name,
    p.category,
    SUM(oi.quantity) AS total_quantity_sold,
    RANK() OVER (
        PARTITION BY p.category
        ORDER BY SUM(oi.quantity) DESC
    ) AS category_rank
FROM Products p
JOIN Order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_id, p.product_name, p.category
ORDER BY p.category, category_rank;
```

d) CTE & Case:

17.Categorize products as ‘High’, ‘Medium’, or ‘Low’ price using CASE.

```
SELECT
    product_id,
    product_name,
```

```
    category,
    price,
    CASE
        WHEN price >= 1000 THEN 'High'
        WHEN price >= 500 THEN 'Medium'
        ELSE 'Low'
    END AS price_category
FROM Products
ORDER BY category, price DESC;
```

18.Use a CTE to find top 3 customers by spending.

```
WITH CustomerSpending AS (
    SELECT
        c.customer_id,
        c.name,
        SUM(oi.price_each * oi.quantity) AS total_spent
    FROM Customers c
    JOIN Orders o ON c.customer_id = o.customer_id
    JOIN Order_items oi ON o.order_id = oi.order_id
    GROUP BY c.customer_id, c.name
)

SELECT customer_id, name, total_spent
FROM CustomerSpending
ORDER BY total_spent DESC
LIMIT 3;
```

19.Use a CTE with CASE to classify customers by loyalty (based on number of

orders).

```
WITH CustomerOrderCounts AS (
    SELECT
        c.customer_id,
        c.name,
        COUNT(o.order_id) AS order_count
    FROM Customers c
    JOIN Orders o ON c.customer_id = o.customer_id
    GROUP BY c.customer_id, c.name
)

SELECT
    customer_id,
    name,
    order_count,
    CASE
        WHEN order_count >= 10 THEN 'High Loyalty'
        WHEN order_count >= 5 THEN 'Medium Loyalty'
        ELSE 'Low Loyalty'
    END AS loyalty_level
FROM CustomerOrderCounts
ORDER BY order_count DESC;
```

20.Find monthly revenue growth percentage compared to the previous month.

```
WITH MonthlyRevenue AS (
    SELECT
        DATE_TRUNC('month', o.order_date) AS month,
        SUM(oi.price_each * oi.quantity) AS revenue
```

```
    FROM Orders o
    JOIN Order_items oi ON o.order_id = oi.order_id
    GROUP BY DATE_TRUNC('month', o.order_date)
),
RevenueGrowth AS (
    SELECT
        month,
        revenue,
        LAG(revenue) OVER (ORDER BY month) AS previous_month_revenue,
        ROUND(
            (revenue - LAG(revenue) OVER (ORDER BY month)) * 100.0 /
            NULLIF(LAG(revenue) OVER (ORDER BY month), 0),
            2
        ) AS growth_percentage
    FROM MonthlyRevenue
)

SELECT * FROM RevenueGrowth
ORDER BY month;
```

21.Find top 2 customers per city based on total spending.

```
WITH CustomerSpending AS (
    SELECT
        c.customer_id,
```

```
    c.name,
    c.city,
    SUM(oi.price_each * oi.quantity) AS total_spent
  FROM Customers c
  JOIN Orders o ON c.customer_id = o.customer_id
  JOIN Order_items oi ON o.order_id = oi.order_id
  GROUP BY c.customer_id, c.name, c.city
),
RankedCustomers AS (
  SELECT *,
      RANK() OVER (PARTITION BY city ORDER BY total_spent DESC) AS city_rank
  FROM CustomerSpending
)
SELECT customer_id, name, city, total_spent, city_rank
FROM RankedCustomers
WHERE city_rank <= 2
ORDER BY city, city_rank;
```

e) Miscellaneous Advanced Joins & Aggregations

22.Find top 3 cities with highest sales revenue including shipper name.

```
WITH CityRevenue AS (
  SELECT
    cu.city,
    s.shipper_name,
    SUM(oi.price_each * oi.quantity) AS total_revenue
  FROM Orders o
  JOIN Customers cu ON o.customer_id = cu.customer_id
```

```
    JOIN Order_items oi ON o.order_id = oi.order_id
    JOIN Shippers s ON o.shipper_id = s.shipper_id
    GROUP BY cu.city, s.shipper_name
)

SELECT city, shipper_name, total_revenue
FROM CityRevenue
ORDER BY total_revenue DESC
LIMIT 3;
```

23.List all orders with customer name, product name, supplier, and shipper.

```
SELECT
    o.order_id,
    c.name AS customer_name,
    p.product_name,
    s.supplier_name,
    sh.shipper_name
FROM Orders o
JOIN Customers c ON o.customer_id = c.customer_id
JOIN Order_items oi ON o.order_id = oi.order_id
JOIN Products p ON oi.product_id = p.product_id
JOIN Suppliers s ON p.supplier_id = s.supplier_id
JOIN Shippers sh ON o.shipper_id = sh.shipper_id
ORDER BY o.order_id;
```

24.Show total sales per supplier along with average order value.

```
SELECT
```

```
    s.supplier_id,
    s.supplier_name,
    SUM(oi.price_each * oi.quantity) AS total_sales,
    ROUND(
        SUM(oi.price_each * oi.quantity) * 1.0 / COUNT(DISTINCT o.order_id),
        2
    ) AS average_order_value
FROM Suppliers s
JOIN Products p ON s.supplier_id = p.supplier_id
JOIN Order_items oi ON p.product_id = oi.product_id
JOIN Orders o ON oi.order_id = o.order_id
GROUP BY s.supplier_id, s.supplier_name
ORDER BY total_sales DESC;
```

25.Show product categories that contributed more than 30% of total sales revenue.

```
WITH CategoryRevenue AS (
    SELECT
        p.category,
        SUM(oi.price_each * oi.quantity) AS category_sales
    FROM Order_items oi
    JOIN Products p ON oi.product_id = p.product_id
    GROUP BY p.category
),
TotalRevenue AS (
    SELECT SUM(category_sales) AS total_sales
    FROM CategoryRevenue
)
```

```
SELECT
    cr.category,
    cr.category_sales,
    ROUND(cr.category_sales * 100.0 / tr.total_sales, 2) AS percentage_contribution
FROM CategoryRevenue cr
CROSS JOIN TotalRevenue tr
WHERE cr.category_sales * 100.0 / tr.total_sales > 30
ORDER BY percentage_contribution DESC;
```